

Parameterized testing

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Example: Username validation

Consider the username validation exercise from Chapter 1:

```
public static boolean isValidUsername(String username) {  
    if (username == null || username.isEmpty() || username.contains(" ")) {  
        return false;  
    }  
    return username.length() >= 3;  
}
```

Example: Repetitive validation test

Recall that the tests looked like this:

```
@Test
void isValidUsername_returnsFalse() {
    String username = "____";

    boolean actual = isValidUsername(username);

    assertFalse(actual);
}
```

We don't have to write 3 of these tests!

¹ https://en.wikipedia.org/wiki/Don%27t_repeat_yourself

Parameterized tests: syntax

Instead of `@Test`, we use `@ParameterizedTest`

It is imported from:

```
import org.junit.jupiter.params.ParameterizedTest;
```

We also need one further annotation: `@ValueSource`.

It is imported from:

```
import org.junit.jupiter.params.provider.ValueSource;
```

Parameterized tests: example test

```
@ParameterizedTest
@ValueSource(strings = {"", "jane doe"}) // NOTE: It's "strings", not String
void isValidUsername_returnsFalse(String username) { // Tests can take arguments
    boolean actual = isValidUsername(username);

    assertFalse(actual);
}
```

JUnit executes this as two tests:

```
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
All tests passed!
```

@ValueSource limitations

`@ValueSource` is limited to certain types.

```
@ValueSource(bytes = {1, 2, 3, 42})
@ValueSource(shorts = {100, 200, 300})
@ValueSource(ints = {1, 2, 3, 42})
@ValueSource(longs = {1000L, 2000L, 3000L})
@ValueSource(floats = {1.0f, 3.14159f, 2.71828f})
@ValueSource(doubles = {3.14, 2.71828, 1.4142})
@ValueSource(chars = {'a', 'b', 'c', 'Z'})
@ValueSource(booleans = {true, false})
@ValueSource(strings = {"Hello", "JUnit", "5", "Parameter"})
@ValueSource(classes = {String.class, Integer.class, ValueSourceExamples.class})
```

1

<https://junit.org/junit5/docs/5.7.1/api/org.junit.jupiter.params/org/junit/jupiter/params/provider/ValueSource.html>

@NullSource

We cannot pass `null` to `@ValueSource` as it will not compile:

```
@ParameterizedTest
@ValueSource(strings = {"", "jane doe", null}) // Does not compile
void isValidUsername_returnsFalse(String username) {
    boolean actual = isValidUsername(username);

    assertFalse(actual);
}
```

Instead use `@NullSource` .

Running the full ParameterizedTest

If we pass `null` to `@ValueSource`, it will not compile.

```
@ParameterizedTest
@NullSource // Adds a null input test case
@ValueSource(strings = {"", "jane doe"})
void isValidUsername_returnsFalse(String username) {
    boolean actual = isValidUsername(username);

    assertFalse(actual);
}
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
```

```
All tests passed!
```


Multiple arguments per test

Another limitation of `@ValueSource` - can only pass one value per test.

Consider the following method:

```
int countLetters(String input) {  
    if (input != null) {  
        return input.length();  
    }  
    return 0;  
}
```

@CsvSource

For multiple values, use `@CsvSource` (CSV = comma separated values):

```
@ParameterizedTest
@CsvSource({"Hello World, 11", "DataCamp, 8", "'", 0", "", 0"})
void countLetters_countsLetters(String input, int expected) {
    int actual = countLetters(input);

    assertEquals(expected, actual);
}
```

Note `", 0"` is inputted as null, 0. To input an empty string, use `"', 0"` in JUnit 5.10

`@CsvSource` has similar type limitations as `@ValueSource`.

Note: Importing Source annotation

All `@___Source` annotations can be imported from:

```
import org.junit.jupiter.params.provider.*;
```

Let's practice!

INTRODUCTION TO TESTING IN JAVA

Additional annotations

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Prerequisite: Object creation with .of()

Consider:

```
List<Integer> newList = new ArrayList<Integer>();  
newList.add(10);  
newList.add(20);  
newList.add(30);
```

Quicker and shorter way instead:

```
List<Integer> newList = List.of(10, 20, 30);
```

- Works the same for `Set` and `Map` from Java Collections
- Objects created with `.of()` are typically immutable (can't be changed)

Passing objects to @ParameterizedTest

Consider the following class:

```
class Person {  
    String firstName;  
    String lastName;  
    public Person(String firstName, String lastName) {  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
    public String fullName(Person person) {  
        return firstName + " " + lastName;  
    }  
}
```


Arguments class

We use `Arguments` to pass multiple args of any type.

`Arguments` - can contain any number of objects of any kind.

For example:

```
Arguments.of(new Person("Monty", "Python"), "Monty Python");
```

We can now use it in the next `@ParameterizedTest` type.

@MethodSource

`@MethodSource` allows us to pass any object to a test.

The test looks like:

```
@ParameterizedTest
@MethodSource("provideNames")
void testFullName(Person person, String expectedFullName) {
    assertEquals(person.fullName(person), expectedFullName);
}
```

@MethodSource provider method

Method for @MethodSource :

```
private static List<Arguments> provideNames() {  
    List<Arguments> args = new ArrayList<>();  
    args.add(Arguments.of(new Person("Robert", "Martin"), "Robert Martin"));  
    args.add(Arguments.of(new Person("Heinz", "Kabutz"), "Heinz Kabutz"));  
  
    return args;  
}
```

Note: method needs to be static

Note: many permitted return types; List<Arguments> is the simplest

¹ <https://junit.org/junit5/docs/5.2.0/api/org/junit/jupiter/params/provider/MethodSource.html>

@MethodSource provider method

Method for @MethodSource :

```
private static List<Arguments> provideNames() {  
    return List.of(  
        Arguments.of(new Person("John", "Doe"), "John Doe"),  
        Arguments.of(new Person("Jane", "Doe"), "Jane Doe"),  
        Arguments.of(new Person("Alice", "Bob"), "Alice Bob"));  
}
```

What if we need more than arguments?

Consider the databases test:

```
@Test
void process_savesToInfoStore_whenInfoMessage() {
    InfoStore infoStore = mock(InfoStore.class);
    ErrorStore errorStore = mock(ErrorStore.class);
    MessageProcessor messageProcessor = new MessageProcessor(infoStore, errorStore);

    messageProcessor.saveMessage("[INFO] Process started.");

    verify(infoStore).save("[INFO] Process started.");
    verifyNoInteractions(errorStore);
}
```

@BeforeEach annotation

We can use `@BeforeEach` annotation to create a method that executes before each test:

```
import org.junit.jupiter.api.BeforeEach;
```

To use it, first declare objects as fields:

```
class MessageProcessorTest {  
  
    private InfoStore infoStore;  
    private ErrorStore errorStore;  
    private MessageProcessor messageProcessor;  
}
```

Method for @BeforeEach

Create each object in a separate method:

```
@BeforeEach
void setUp() {
    this.infoStore = mock(InfoStore.class);
    this.errorStore = mock(errorStore.class);
    this.messageProcessor = new MessageProcessor(infoStore, errorStore);
}
```

Shortened test

Test class then becomes:

```
@Test
void process_savesToInfoStore_whenInfoMessage() {
    messageProcessor.process("[INFO] Process started.");

    verify(infoStore).save("[INFO] Process started.");
    verifyNoInteractions(errorStore);
}
```


@BeforeEach full flow

All together:

```
class MessageProcessorTest {  
    private InfoStore infoStore; // Declare fields  
  
    @BeforeEach  
    void setUp() {  
        this.infoStore = mock(InfoStore.class); // Initialize fields  
    }  
  
    @Test  
    void process_savesToInfoStore_whenInfoMessage() {  
        // Use fields  
    }  
}
```


Let's practice!

INTRODUCTION TO TESTING IN JAVA

Test-driven development

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Test-Driven Development

So far we have only written tests after writing production code.

- Test-Driven Development (TDD) is a design technique where tests guide the design
- TDD is done by writing tests before writing any production code
- Why do TDD?

Benefits of TDD

- Developing with TDD leads to fulfilling the project requirements better
- Instant verification - any code we write already has tests
- Coding based on tests leads to cleaner, better-structured code

The Laws of TDD

- 1. Must write a failing test before writing any production code*
- 2. Must write the shortest test possible*
- 3. Must not write more production code than is sufficient to make the test pass*

Applying these in sequence = one cycle of TDD

Development happens through repeated cycles of TDD

¹ <https://blog.cleancoder.com/uncle-bob/2014/12/17/TheCyclesOfTDD.html>

Testing for palindromes

- Suppose we want to write a method that checks if an integer is a palindrome
- Examples of palindromic integers: 12321, 0, 3333

Write a failing test before writing any code

```
@Test
public void isPalindrome_nonPalindromicInt_returnsFalse() {
    int number = 1234;

    assertFalse(isPalindrome(number)); // isPalindrome() is not yet written
}
```

A test is like a note we leave to our future selves that reminds us of the project requirements.

Write minimal code for the test to pass

```
public boolean isPalindrome(int num) {  
    return false; // Write minimal code that fulfills the requirements of the test.  
}
```

Test now passes. We can move on to the next cycle of TDD.

Write another test

New test - new requirement for the feature:

```
@Test
public void isPalindrome_PalindromicInt_ReturnsTrue() {
    int number = 1234321; // Now write a more complex test case.

    assertTrue(isPalindrome(number)); // At this point isPalindrome() will fail.
}
```

Implement the method

```
public boolean isPalindrome(int num) {  
    int inverted = 0;  
    while (num != 0) {  
        // At every iteration take the last digit of the number using % 10  
        // and add it to inverted * 10.  
        inverted = inverted * 10 + num % 10;  
        // Now that we have processed the last digit, discard it using / 10.  
        num = num / 10;  
    }  
    return inverted == num;  
}
```

Bugfixing with TDD

TDD is great for fixing bugs!

The method we just wrote returns true for `number = -121` . This is not correct!

We write a unit test to illustrate the bug:

```
@Test
public void isPalindrome_NegativeNumber_ReturnsFalse() {
    int number = -121; // Not a palindrome as -121 is not the same as 121-.

    assertFalse(isPalindrome(number));
}
```

Bugfixing with TDD

```
public boolean isPalindrome(int num) {  
    if (num < 0) {  
        return false; // We add a clause to return false for negative values.  
    }  
    int inverted = 0;  
    while (num != 0) {  
        inverted = inverted * 10 + num % 10;  
        num = num / 10;  
    }  
    return inverted;  
}
```

Let's practice!

INTRODUCTION TO TESTING IN JAVA

Wrap-up

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Chapter 1 Recap

Chapter 1.1:

- Introduced the concept of software testing
- Discussed the benefits of testing
- Reviewed some edge cases we should always test

Chapter 1.2:

- Introduced testing with JUnit
- Learned about JUnit 5 syntax like `@Test`
- Practiced `assertEquals()`

Chapter 1.3:

- Learned to define unit testing
- Studied additional assert statements: `assertTrue()`, `assertInstanceOf()`, `assertNull()`

Chapter 2 Recap

Chapter 2.1:

- Learned about software dependencies
- Defined integration testing as testing how components work together
- Practiced using the European Central Bank rates

Chapter 2.2:

- Discovered mocks as a way to control test conditions
- Studied Mockito and its syntax: `mock()` , `when()` , `.thenReturn()`

Chapter 2.3:

- Learned `verify()` to track mock interactions
- Practiced using database examples

Chapter 3 Recap

Chapter 3.1:

- Used `@ParameterizedTest` to shorten test classes and reduce repetition
- Practiced `@ValueSource` and `@CsvSource`

Chapter 3.2:

- Discovered ways to pass more complex arguments to tests using `@MethodSource` and `Arguments.of()`
- Encountered the `@BeforeEach` annotation

Chapter 3.3:

- Practiced thinking with tests first
- Defined Test-Driven Development as the practice of writing tests before implementation
- Learned to fix bugs with TDD

Further learning:

Interesting talks:

- [The Joy of Testing](#) by Victor Rentea - one of the most fun talks on testing out there
- [Are Your Tests Slowing You Down?](#) by Trisha Gee
- [Test Driven Design Insights](#) by Victor Rentea

Advanced testing study:

- [Property-based testing with jqwik](#) - a new testing approach + a library that finds edge cases for you.
- [Behavior-Driven Development](#) - like TDD, but bigger and more advanced.
- [Intro to performance testing](#) - performance is important for any Java application.

Congratulations!

INTRODUCTION TO TESTING IN JAVA